

EXPLICACAO DO APLICATIVO CARDAPIO

OBJETIVO DO APP

O aplicativo mostra um cardapio de restaurante. Quando abre, ele tenta buscar os dados de um JSON remoto. Esse JSON vem do servidor local na pasta servidor-cardapio. Cada item possui id, nome, preco e url da imagem. Se a carga online funcionar, o app salva os dados no SQLite e salva as imagens no sistema de arquivos. Se o servidor estiver desligado ou o app estiver offline, ele le os dados locais e mostra o preco como "a consultar".

ARQUIVOS PRINCIPAIS

MainActivity.kt: controla a tela, busca o JSON, baixa imagens, chama o SQLite e monta os cards visuais.

CardapioDBHelper.kt: cria e manipula o banco SQLite.

activity_main.xml: tem o ScrollView e o LinearLayout onde os itens sao adicionados.

AndroidManifest.xml: declara permissao de internet e permissao para verificar estado de rede.

cardapio.json: arquivo remoto simulado com os dados do cardapio.

FLUXO GERAL

1. A tela abre e executa onCreate.
2. O app pega o LinearLayout do XML e prepara o banco SQLite.
3. A funcao isOnline verifica se existe conexao.
4. Se houver conexao, o app chama carregarDadosOnline.
5. carregarDadosOnline baixa o JSON, percorre os itens, baixa as imagens e salva tudo localmente.
6. Depois mostra os cards com nome, foto e preco.
7. Se houver erro ou estiver offline, o app chama carregarDadosOffline.
8. carregarDadosOffline le o SQLite e mostra as imagens salvas.
9. No modo offline, o preco real nao aparece; aparece "a consultar".

MAINACTIVITY.KT EM ORDEM

package com.example.trab_mobile_m2

Define o pacote da classe. E como a organizacao logica do codigo dentro do projeto.

imports

Os imports trazem classes prontas. Por exemplo: TextView para texto, ImageView para imagem, LinearLayout para organizar componentes, JSONArray para ler JSON, HttpURLConnection para HTTP e File para acessar arquivos.

class CardapioItem

Classe simples criada para representar um item do cardapio dentro do app.

var id: Int = 0

Guarda o identificador do prato. Int significa numero inteiro.

var nome: String = ""

Guarda o nome do prato. String significa texto.

var preco: Double = 0.0

Guarda o preco. Double significa numero com casas decimais.

var imagemLocal: String = ""

Guarda o caminho da imagem salva no celular.

class MainActivity : AppCompatActivity()

Define a tela principal. O simbolo dois pontos indica heranca. A MainActivity herda recursos de AppCompatActivity.

```
private lateinit var containerCardapio: LinearLayout
```

Variavel que vai guardar o LinearLayout da tela. lateinit significa que ela sera inicializada depois.

```
private lateinit var dbHelper: CardapioDBHelper
```

Variavel que acessa o banco SQLite.

```
private val mainHandler = Handler(Looper.getMainLooper())
```

Cria um Handler da thread principal. Ele permite atualizar a interface depois que a thread de internet terminar.

```
private val cardapioUrl = "http://10.0.2.2:8080/cardapio.json"
```

Guarda o endereco do JSON. No emulador Android, 10.0.2.2 aponta para o computador onde o servidor local esta rodando.

```
onCreate
```

E a primeira funcao executada quando a tela abre.

```
super.onCreate(savedInstanceState)
```

Chama o comportamento original da Activity.

```
setContentView(R.layout.activity_main)
```

Liga a tela ao arquivo XML activity_main.

```
containerCardapio = findViewById(R.id.container_cardapio)
```

Procura no XML o LinearLayout com esse id e guarda na variavel.

```
dbHelper = CardapioDBHelper(this)
```

Cria o objeto que manipula o banco SQLite.

```
if (isOnline())
```

Chama a funcao que verifica internet. Se retornar true, carrega online.

```
carregarDadosOnline()
```

Inicia a carga pelo servidor.

```
carregarDadosOffline()
```

Inicia a carga pelo banco local quando nao ha internet.

```
isOnline
```

Verifica se existe uma rede ativa com capacidade de internet.

```
getSystemService(Context.CONNECTIVITY_SERVICE) as ConnectivityManager
```

Pega o servico de conectividade do Android e converte para ConnectivityManager.

```
val network = connectivityManager.activeNetwork
```

Pega a rede ativa atual.

```
if (network == null) return false
```

Se nao existir rede ativa, retorna falso.

```
val activeNetwork = connectivityManager.getNetworkCapabilities(network)
```

Pega as capacidades da rede, como ter acesso a internet.

```
return activeNetwork.hasCapability(NetworkCapabilities.NET_CAPABILITY_INTERNET)
```

Retorna verdadeiro se a rede tiver capacidade de internet.

```
carregarDadosOnline
```

Funcao responsavel por buscar dados no servidor.

```
Thread { ... }.start()
```

Cria uma thread separada. Isso e necessario porque o Android nao permite requisicao HTTP na thread principal da tela.

```
try
```

Tenta executar o codigo online.

```
val jsonString = baixarTexto(cardapioUrl)
```

Baixa o texto do arquivo JSON.

```
val jsonArray = JSONArray(jsonString)
```

Transforma o texto JSON em uma lista JSON.

```
val itens = ArrayList<CardapioItem>()
```

Cria uma lista vazia para armazenar os pratos.

```
for (i in 0 until jsonArray.length())
```

Percorre todos os itens do JSON. Se ha 4 itens, o i passa por 0, 1, 2 e 3.

```
val itemJson = jsonArray.getJSONObject(i)
```

Pega o item JSON da posicao atual.

```
val item = CardapioItem()
```

Cria um prato vazio.

```
item.id = itemJson.getInt("id")
```

Le o id do JSON e coloca no objeto.

```
item.nome = itemJson.getString("nome")
```

Le o nome do JSON.

```
item.preco = itemJson.getDouble("preco")
```

Le o preco do JSON.

```
val urlImagem = itemJson.getString("url_imagem")
```

Le o endereco da imagem.

```
item.imagemLocal = baixarESalvarImagem(...)
```

Baixa a imagem e guarda no objeto o caminho local dela.

```
itens.add(item)
```

Adiciona o prato na lista.

```
salvarCargaLocal(itens)
```

Salva a lista no SQLite.

mainHandler.post

Volta para a thread principal antes de mexer na tela.

exibirCardapioNaTela(itens, true)

Mostra os itens na tela em modo online.

catch (e: Exception)

Se qualquer erro acontecer, cai aqui.

carregarDadosOffline()

Usa os dados locais como plano B.

baixarTexto

Recebe uma URL e retorna o texto baixado.

URL(urlString)

Transforma o texto em objeto URL.

url.openConnection() as HttpURLConnection

Abre uma conexao HTTP.

conexao.requestMethod = "GET"

Define que a requisicao vai buscar dados.

connectTimeout e readTimeout

Definem tempo maximo para conectar e ler resposta.

conexao.responseCode

Pega o codigo HTTP retornado pelo servidor.

if (codigoResposta < 200 || codigoResposta > 299)

Se o codigo nao estiver na faixa de sucesso, lanca erro.

BufferedReader(InputStreamReader(inputStream))

Prepara a leitura do texto recebido.

reader.readText()

Le todo o JSON como texto.

finally { conexao.disconnect() }

Fecha a conexao mesmo se der erro.

baixarESalvarImagem

Recebe a URL da imagem e o nome do arquivo local.

BitmapFactory.decodeStream(inputStream)

Converte os bytes da imagem em Bitmap, formato que o Android entende.

if (bitmap == null)

Se nao conseguiu ler a imagem, lanca erro.

openFileOutput(nomeArquivo, Context.MODE_PRIVATE)

Abre um arquivo privado do app para salvar a imagem.

bitmap.compress(Bitmap.CompressFormat.JPEG, 90, outputStream)
Salva a imagem em JPEG com qualidade 90.

File(filesDir, nomeArquivo).absolutePath
Monta e retorna o caminho completo do arquivo salvo.

salvarCargaLocal
Limpa o banco e grava a nova carga online.

dbHelper.limparBanco()
Remove os registros antigos para evitar duplicidade.

for (i in 0 until itens.size)
Percorre a lista de pratos.

dbHelper.salvarPrato(...)
Salva cada prato no SQLite.

carregarDadosOffline
Le os dados salvos no SQLite.

val db = dbHelper.readableDatabase
Abre o banco para leitura.

val sql = "SELECT ..."
Monta a consulta SQL que busca id, nome, preco e imagem_local.

db.rawQuery(sql, null)
Executa a consulta.

cursor.getColumnIndexOrThrow(...)
Descobre a posicao de cada coluna no resultado.

while (cursor.moveToNext())
Percorre cada linha retornada pelo banco.

cursor.getInt, getString e getDouble
Le os valores da linha atual.

cursor.close() e db.close()
Fecham o cursor e o banco depois da leitura.

exibirCardapioNaTela(itens, false)
Mostra os itens em modo offline.

exibirCardapioNaTela
Limpa a tela, adiciona cabecalho e cria os cards.

containerCardapio.removeAllViews()
Remove tudo que estava na tela antes.

setBackgroundColor e setPadding
Definem cor de fundo e espacamento.

adicionarCabecalho(online)

Mostra o título e o status da carga.

if (lista.isEmpty())

Se não houver dados locais, mostra mensagem de aviso.

criarCardItem(item, online)

Cria o visual de cada prato.

adicionarCabecalho

Cria o título "Cardápio do Restaurante" e a mensagem de status.

TextView(this)

Cria um texto pela programação.

título.text, textSize, typeface, setTextColor e gravity

Configuram texto, tamanho, negrito, cor e alinhamento.

adicionarMensagemVazia

Mostra uma mensagem se o app estiver offline e sem cache local.

criarCardItem

Monta um card com imagem, nome e preço.

LinearLayout(this)

Cria o bloco do card.

card.orientation = LinearLayout.VERTICAL

Organiza os componentes um embaixo do outro.

GradientDrawable

Cria o fundo branco com borda do card.

ImageView(this)

Cria o componente de imagem.

imagem.scaleType = ImageView.ScaleType.CENTER_CROP

Faz a imagem preencher o espaço cortando as sobras.

File(item.imagemLocal)

Aponta para a imagem salva no celular.

if (arquivoFoto.exists())

Se a foto existe, carrega ela.

BitmapFactory.decodeFile(...)

Le a imagem salva.

imagem.setImageBitmap(bitmap)

Mostra a imagem no ImageView.

nome.text = item.nome

Mostra o nome do prato.

if (online)

Decide se o preço real será mostrado.

String.format(Locale.forLanguageTag("pt-BR"), "R\$ %.2f", item.preco)

Formata o preço com duas casas decimais.

else preco.text = "a consultar"

No offline, atende ao requisito e não mostra o preço.

return card

Devolve o card pronto para ser colocado na tela.

dp

Converte valores para dp, unidade visual usada pelo Android para telas com densidades diferentes.

CARDAPIODBHELPER.KT

Essa classe cuida do banco SQLite.

SQLiteOpenHelper

Classe do Android usada para criar e atualizar bancos SQLite.

DATABASE_NAME = "cardapio.db"

Nome do arquivo do banco.

DATABASE_VERSION = 2

Versão atual do banco. Foi aumentada porque a coluna imagem_local foi adicionada.

TABLE_NAME = "pratos"

Nome da tabela.

COLUMN_ID, COLUMN_NOME, COLUMN_PRECO, COLUMN_IMAGEM_LOCAL

Constantes com os nomes das colunas.

onCreate(db)

Executa quando o banco é criado pela primeira vez.

CREATE TABLE pratos

Cria a tabela com id, nome, preço e imagem_local.

onUpgrade

Executa quando a versão do banco muda.

DROP TABLE IF EXISTS

Remove a tabela antiga.

onCreate(db)

Cria a tabela nova.

salvarPrato

Recebe id, nome, preço e imagemLocal e salva no banco.

ContentValues

Estrutura usada para mandar valores para o SQLite.

`cv.put(...)`

Coloca cada valor em sua coluna.

`insertWithOnConflict(... CONFLICT_REPLACE)`

Insere o registro. Se ja existir o mesmo id, substitui.

`limparBanco`

Executa DELETE FROM pratos para remover registros antigos.

ANDROIDMANIFEST.XML

INTERNET permite acessar o JSON e as fotos.

ACCESS_NETWORK_STATE permite verificar se existe conexao.

usesCleartextTraffic true permite HTTP sem HTTPS no servidor local.

networkSecurityConfig aponta para a configuracao que libera trafego HTTP.

MainActivity e marcada como tela inicial do app.

ACTIVITY_MAIN.XML

ScrollView permite rolar a tela quando houver muitos pratos.

LinearLayout container_cardapio e o espaco onde os cards sao adicionados por codigo.

CARDAPIO.JSON

E o servico remoto simulado. Possui 4 registros, cumprindo o minimo pedido. Cada registro tem id, nome, preco e url_imagem.

COMO DEMONSTRAR

1. Abrir a pasta servidor-cardapio.
2. Rodar `npm run http-server -p 8080`.
3. Abrir o app no emulador.
4. Mostrar que ele carrega nome, preco e foto.
5. Fechar o servidor.
6. Abrir o app novamente.
7. Mostrar que os dados continuam aparecendo pelo SQLite e pelas imagens locais.
8. Mostrar que no modo offline o preco aparece como "a consultar".

FRASE RESUMO PARA APRESENTAR

O app possui dois fluxos. No fluxo online, ele baixa o JSON remoto, le os pratos, baixa as fotos e salva tudo localmente. No fluxo offline, ele le os dados do SQLite, usa as imagens salvas no sistema de arquivos e esconde o preco, exibindo "a consultar".